

MÔN HỌC

ARTIFICIAL INTELLIGENCE

HỌC KỲ 2533 NĂM HỌC 2025 - 2026

ĐỀ TÀI:

PHÂN LOẠI MALWARE

Giảng Viên Hướng Dẫn: TS. Nguyễn Quốc Minh
Người Thực Hiện: 22302421 - Nguyễn Phúc Minh Châu
22205801 - Huỳnh Thế Bảo
22402742 - Nguyễn Phước Mai Bảo

Thành phố Hồ Chí Minh, ngày 26 tháng 06 năm 2026

MỤC LỤC

MỤC LỤC	2
NỘI DUNG	3
1. GIỚI THIỆU	3
1.1. Bối cảnh bài toán phát hiện mã độc (Malware Detection)	3
1.2. Tập dữ liệu Microsoft Malware Prediction	3
1.3. Mục tiêu đề tài	4
2. TIỀN XỬ LÝ DỮ LIỆU VÀ LỰA CHỌN ĐẶC TRƯNG	4
2.1. Lấy mẫu dữ liệu (Data Sampling)	4
2.2. Mã hóa dữ liệu phân loại (Label Encoding)	5
2.3. Lọc đặc trưng có phương sai thấp (Variance Threshold)	6
2.4. Phân tách tập dữ liệu (Train-Test Split)	7
2.5. Mất cân bằng dữ liệu (Class Imbalance)	7
3. XÂY DỰNG MÔ HÌNH HỌC MÁY	7
3.1. Lý do lựa chọn mô hình	7
3.2. Cây quyết định (Decision Tree Classifier)	8
3.2.1. Nguyên lý hoạt động	8
3.2.2. Chỉ số Gini (Gini Impurity)	8
3.2.3. Entropy và Information Gain	8
3.2.4. Ưu điểm và hạn chế	9
3.2.4. Ưu điểm và hạn chế	9
3.3. Rừng ngẫu nhiên (Random Forest Classifier)	9
3.3.1. Tổng quan	9
3.3.2. Lấy mẫu Bootstrap (Bootstrap Sampling)	10
3.3.3. Lựa chọn thuộc tính ngẫu nhiên (Feature Randomness)	10
3.3.4. Bỏ phiếu đa số (Majority Voting)	10
3.3.5. Xử lý mất cân bằng dữ liệu	10
3.3.6. Độ phức tạp tính toán và khả năng song song hóa	11
4. Kết Quả	11

NỘI DUNG

1. GIỚI THIỆU

1.1. Bối cảnh bài toán phát hiện mã độc (Malware Detection)

Sự phát triển mạnh mẽ của công nghệ thông tin cùng với quá trình chuyển đổi số đã làm gia tăng đáng kể các nguy cơ mất an toàn thông tin. Trong đó, mã độc (Malware) là một trong những mối đe dọa phổ biến nhất, có khả năng gây mất dữ liệu, gián đoạn hoạt động hệ thống và ảnh hưởng đến quyền riêng tư của người dùng.

Các phương pháp phát hiện mã độc truyền thống chủ yếu dựa trên chữ ký (Signature-based Detection). Mặc dù hiệu quả đối với các mẫu mã độc đã biết, phương pháp này gặp nhiều hạn chế khi đối mặt với các biến thể mới hoặc các cuộc tấn công chưa từng xuất hiện trước đó (Zero-day Attacks).

Trong những năm gần đây, học máy (Machine Learning) đã trở thành một hướng tiếp cận tiềm năng trong lĩnh vực an ninh mạng. Thay vì dựa trên chữ ký nhận dạng, mô hình học máy khai thác các đặc trưng về cấu hình hệ thống, phần cứng và trạng thái bảo mật của thiết bị để dự đoán khả năng thiết bị bị nhiễm mã độc.

1.2. Tập dữ liệu Microsoft Malware Prediction

Đề tài sử dụng tập dữ liệu Microsoft Malware Prediction được công bố trên nền tảng Kaggle. Đây là một trong những bộ dữ liệu quy mô lớn trong lĩnh vực phát hiện mã độc, bao gồm khoảng 8,9 triệu bản ghi huấn luyện với 83 thuộc tính mô tả đặc điểm của thiết bị.

Các thuộc tính trong bộ dữ liệu phản ánh nhiều khía cạnh khác nhau của hệ thống, bao gồm:

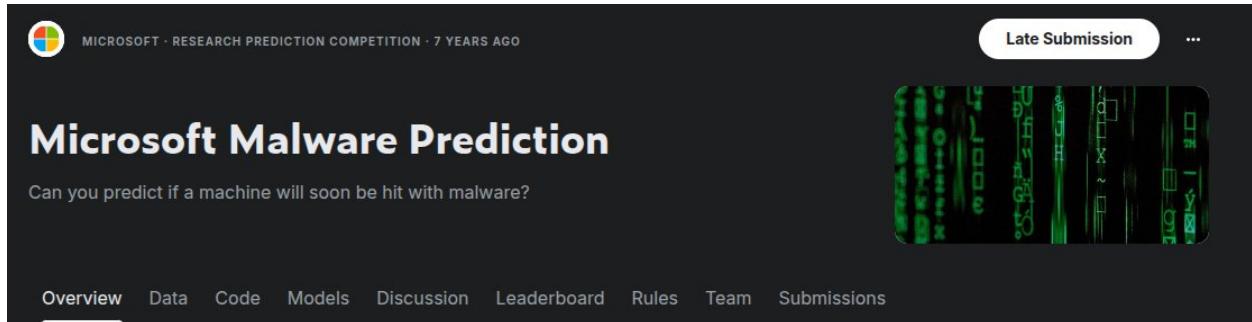
- Thông tin hệ điều hành (Operating System).
- Cấu hình phần cứng (Hardware Configuration).
- Trạng thái phần mềm bảo mật (Security Software Status).
- Thông tin khu vực địa lý (Geographical Information).
- Các thuộc tính liên quan đến thiết bị và môi trường sử dụng.

Biến mục tiêu (Target Variable) của bộ dữ liệu là HasDetections, thể hiện trạng thái phát hiện mã độc trên thiết bị.

Giá trị của biến được quy ước như sau:

- 0: Không phát hiện mã độc trên thiết bị.
- 1: Phát hiện mã độc trên thiết bị.

Do biến mục tiêu chỉ gồm hai giá trị, bài toán được xây dựng dưới dạng bài toán phân loại nhị phân (Binary Classification). Nhiệm vụ của mô hình là dự đoán khả năng thiết bị thuộc lớp có phát hiện mã độc hay không dựa trên các đặc trưng đầu vào.



1.3. Mục tiêu đề tài

Mục tiêu của đề tài là xây dựng và đánh giá mô hình học máy nhằm dự đoán khả năng thiết bị bị nhiễm mã độc dựa trên các đặc trưng hệ thống.

Hai mô hình được lựa chọn để so sánh gồm:

- Cây quyết định (Decision Tree Classifier): mô hình cơ sở (Baseline Model).
- Rừng ngẫu nhiên (Random Forest Classifier): mô hình học máy kết hợp (Ensemble Learning Model).

Thông qua việc so sánh các chỉ số đánh giá hiệu năng, nghiên cứu xác định mức độ cải thiện mà kỹ thuật học máy kết hợp mang lại so với mô hình cây quyết định đơn lẻ.

2. TIỀN XỬ LÝ DỮ LIỆU VÀ LỰA CHỌN ĐẶC TRƯNG

2.1. Lấy mẫu dữ liệu (Data Sampling)

Tập dữ liệu gồm hơn 8,9 triệu bản ghi, đòi hỏi tài nguyên tính toán lớn để xử lý và huấn luyện mô hình. Để đảm bảo khả năng thực nghiệm trên môi trường phần cứng thông thường, đề tài thực hiện kỹ thuật lấy mẫu phân tầng theo nhãn (Stratified Sampling).

Sau quá trình lấy mẫu, tập dữ liệu được giảm xuống còn 20.002 bản ghi và vẫn giữ nguyên 83 thuộc tính ban đầu. Phương pháp này giúp duy trì tỷ lệ phân bố giữa các lớp nhãn, đồng thời đảm bảo tính đại diện của dữ liệu đối với bài toán phân loại.

```

1 chunks = []
2 chunk_size = 500000
3 frac = 20000 / 8921483
4
5 for chunk in pd.read_csv('microsoft-malware-prediction/train.csv', chunksize=chunk_size, low_memory=False):
6     sampled_chunk = chunk.sample(frac=frac, random_state=42)
7     chunks.append(sampled_chunk)
8
9 df = pd.concat(chunks, ignore_index=True)
10 print(df.shape)
11 df

```

(20002, 83)

2.2. Mã hóa dữ liệu phân loại (Label Encoding)

Nhiều thuộc tính trong bộ dữ liệu được biểu diễn dưới dạng chuỗi ký tự, chẳng hạn như phiên bản hệ điều hành hoặc loại sản phẩm bảo mật. Các thuật toán học máy không thể xử lý trực tiếp dữ liệu dạng văn bản, do đó cần thực hiện chuyển đổi sang dạng số.

Đề tài sử dụng kỹ thuật Mã hóa nhãn (Label Encoding) để ánh xạ mỗi giá trị phân loại thành một giá trị số nguyên tương ứng.

Ưu điểm của phương pháp này là:

- Giảm kích thước dữ liệu đầu vào.
- Hạn chế hiện tượng gia tăng số chiều dữ liệu.
- Phù hợp với các mô hình dựa trên cây như Cây quyết định (Decision Tree) và Rừng ngẫu nhiên (Random Forest).

Mặc dù Label Encoding giúp chuyển đổi dữ liệu phân loại sang dạng số một cách hiệu quả, phương pháp này có thể tạo ra quan hệ thứ tự giả giữa các giá trị phân loại. Điều này có thể ảnh hưởng đến hiệu quả của một số thuật toán dựa trên khoảng cách hoặc hồi quy tuyến tính.

Tuy nhiên, đối với các mô hình dựa trên cây như Cây quyết định (Decision Tree) và Rừng ngẫu nhiên (Random Forest), việc phân chia dữ liệu được thực hiện thông qua các điều kiện tại các nút thay vì dựa trên khoảng cách giữa các giá trị. Do đó, ảnh hưởng của hạn chế này là không đáng kể và Label Encoding vẫn là lựa chọn phù hợp cho nghiên cứu.

```
1 # Encode categorical variables
2 encoders = {}
3 for col in cat_cols:
4     le = LabelEncoder()
5     df_clean[col] = le.fit_transform(df_clean[col].astype(str))
6     encoders[col] = le
7
8 print("Categorical encoding done (Label Encoding).")
9
10 ##### Imbalance check (actual resampling happens after split, train-only)
11 print(f"\nClass imbalance ratio:\n{df_clean['HasDetections'].value_counts(normalize=True)}")
12 df_clean
```

Categorical encoding done (Label Encoding).

Class imbalance ratio:
HasDetections
1 0.5012
0 0.4988
Name: proportion, dtype: float64

2.3. Loại đặc trưng có phương sai thấp (Variance Threshold)

Trong tập dữ liệu thực tế thường tồn tại các thuộc tính gần như không thay đổi giữa các quan sát. Những thuộc tính này chứa rất ít thông tin phân biệt và ít đóng góp vào quá trình học của mô hình.

Đề tài áp dụng phương pháp Ngưỡng phương sai (Variance Threshold) nhằm loại bỏ các đặc trưng có phương sai thấp.

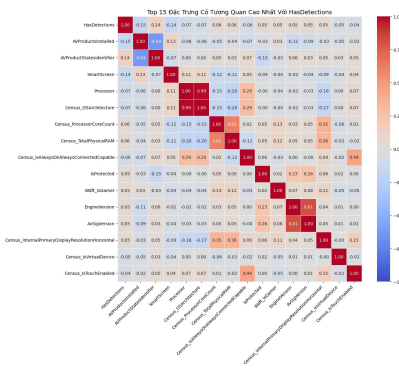
Phương sai của một thuộc tính được tính theo công thức:

$$Var(X) = E[(X - \mu)^2]$$

Trong đó:

- X là giá trị của thuộc tính.
- μ là giá trị trung bình của thuộc tính.

Sau bước xử lý này, số lượng thuộc tính giảm từ 83 xuống còn 75 thuộc tính, giúp giảm nhiễu và tăng hiệu quả huấn luyện.



```
1 # Split labels here for anti Data Leaking
2 X = df_clean.drop(columns=['HasDetections'])
3 y = df_clean['HasDetections']
4 # Remove near-constant / low-variance features
5 selector = VarianceThreshold(threshold=0.01)
6 selector.fit(X)
7 selected_cols = X.columns[selector.get_support()]
8 removed_cols = X.columns[-selector.get_support()]
9
10 print(f"Removed {len(removed_cols)} low-variance columns: {list(removed_cols)}")
11 X = X[selected_cols]
12 print(f"Remaining features in X: {X.shape[1]}")
```

Removed 8 low-variance columns: ['IsBeta', 'AutoSampleOptin', 'SMode', 'UacLUsenable', 'Census_DeviceFamily', 'Census_IsPortableOperatingSystem', 'Census_IsFlightDisabled', 'Census_IsVirtualDevice']
Remaining features in X: 66

2.4. Phân tách tập dữ liệu (Train-Test Split)

Để đánh giá khách quan khả năng dự đoán của mô hình, dữ liệu được chia thành:

- Tập huấn luyện (Training Set): dùng để xây dựng mô hình.
- Tập kiểm thử (Test Set): dùng để đánh giá hiệu năng sau khi huấn luyện.

Việc phân tách dữ liệu giúp hạn chế hiện tượng quá khớp (Overfitting) và phản ánh tốt hơn khả năng tổng quát hóa của mô hình trên dữ liệu chưa từng xuất hiện.

```
1 X_train, X_val, y_train, y_val = train_test_split(
2     X, y, test_size=0.2, stratify=y, random_state=42
3 )
4 print(f"Train shape: {X_train.shape} | Val shape: {X_val.shape}")
5 print(f"Train label dist:\n{y_train.value_counts(normalize=True)}")
6 print(f"Val label dist:\n{y_val.value_counts(normalize=True)}")
```

Train shape: (16001, 66) | Val shape: (4001, 66)
 Train label dist:
 HasDetections
 1 0.501219
 0 0.498781
 Name: proportion, dtype: float64
 Val label dist:
 HasDetections
 1 0.501125
 0 0.498875

2.5. Mất cân bằng dữ liệu (Class Imbalance)

Trong các bài toán phân loại thực tế, số lượng mẫu giữa các lớp thường không hoàn toàn cân bằng. Khi đó, mô hình có xu hướng ưu tiên dự đoán lớp xuất hiện nhiều hơn, dẫn đến suy giảm khả năng nhận diện các mẫu thuộc lớp còn lại.

Để giảm ảnh hưởng của hiện tượng này, đề tài sử dụng cơ chế cân bằng trọng số lớp thông qua tham số: **class_weight = 'balanced'**

Trong quá trình huấn luyện mô hình, cơ chế này tự động điều chỉnh trọng số của từng lớp dựa trên số lượng mẫu tương ứng trong tập dữ liệu. Các lớp có số lượng mẫu ít hơn sẽ được gán trọng số cao hơn, giúp mô hình chú ý nhiều hơn đến các trường hợp dễ bị bỏ sót trong quá trình học.

Việc cân bằng trọng số góp phần cải thiện khả năng phân loại và nâng cao độ tin cậy của các chỉ số đánh giá như Độ chuẩn xác (Precision), Độ thu hồi (Recall) và Chỉ số F1 (F1-score).

3. XÂY DỰNG MÔ HÌNH HỌC MÁY

3.1. Lý do lựa chọn mô hình

Đề tài lựa chọn hai mô hình học máy thuộc nhóm thuật toán dựa trên cây gồm Cây quyết định (Decision Tree) và Rừng ngẫu nhiên (Random Forest) để thực hiện so sánh.

Cây quyết định được sử dụng làm mô hình cơ sở (Baseline Model) nhờ cấu trúc đơn giản, dễ triển khai và dễ giải thích kết quả dự đoán. Trong khi đó, Rừng ngẫu nhiên là phiên bản mở rộng sử dụng kỹ thuật học máy kết hợp (Ensemble Learning), được kỳ vọng cải thiện khả năng tổng quát hóa và giảm hiện tượng quá khớp thường gặp ở cây quyết định đơn lẻ.

Việc lựa chọn hai mô hình này giúp đánh giá rõ ràng mức độ cải thiện về hiệu năng khi áp dụng kỹ thuật học máy kết hợp trên cùng một tập dữ liệu và cùng một quy trình tiền xử lý.

3.2. Cây quyết định (Decision Tree Classifier)

3.2.1. Nguyên lý hoạt động

Cây quyết định là mô hình phân loại dựa trên cấu trúc dạng cây. Quá trình xây dựng cây được thực hiện bằng cách liên tục phân chia dữ liệu thành các tập con nhỏ hơn dựa trên các điều kiện của thuộc tính.

Mỗi nút trong cây đại diện cho một phép kiểm tra điều kiện, trong khi các lá cây (Leaf Nodes) đại diện cho kết quả phân loại cuối cùng.

3.2.2. Chỉ số Gini (Gini Impurity)

Chỉ số Gini được sử dụng để đánh giá mức độ hỗn hợp của dữ liệu tại một nút.

Công thức:

$$Gini = 1 - \sum (p_i^2)$$

Trong đó: p_i là tỷ lệ mẫu thuộc lớp thứ i .

Giá trị Gini càng nhỏ thì nút càng thuần nhất.

3.2.3. Entropy và Information Gain

Entropy đo lường mức độ hỗn loạn của dữ liệu.

Công thức:

$$Entropy = -\sum (p_i \times \log_2(p_i))$$

Từ Entropy, thuật toán xác định thuộc tính phân chia tốt nhất thông qua Information Gain.

$$Information\ Gain = Entropy(Parent) - \sum [(|Child| / |Parent|) \times Entropy(Child)]$$

Thuộc tính có Information Gain lớn nhất sẽ được lựa chọn để phân chia dữ liệu.

3.2.4. Ưu điểm và hạn chế

Ưu điểm:

- Dễ giải thích và trực quan.
- Không yêu cầu dữ liệu tuân theo phân phối chuẩn.
- Có khả năng mô hình hóa các mối quan hệ phi tuyến.

Hạn chế:

- Dễ xảy ra hiện tượng quá khớp (Overfitting).
- Nhạy cảm với nhiễu dữ liệu.
- Kết quả có thể thay đổi đáng kể khi dữ liệu đầu vào thay đổi.

Training Decision Tree...

DecisionTreeClassifier	
Parameters	
criteria	'entropy'
splitter	'best'
max_depth	15
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	None
random_state	42
max_leaf_nodes	None
min_impurity_decrease	0.0
class_weight	'balanced'
ccp_alpha	0.0
monotonic_cst	None

3.2.4. Ưu điểm và hạn chế

Ưu điểm:

- Dễ giải thích và trực quan.
- Không yêu cầu dữ liệu tuân theo phân phối chuẩn.
- Có khả năng mô hình hóa các mối quan hệ phi tuyến.

Hạn chế:

- Dễ xảy ra hiện tượng quá khớp (Overfitting).
- Nhạy cảm với nhiễu dữ liệu.
- Kết quả có thể thay đổi đáng kể khi dữ liệu đầu vào thay đổi.

3.3. Rừng ngẫu nhiên (Random Forest Classifier)

3.3.1. Tổng quan

Rừng ngẫu nhiên là mô hình học máy kết hợp (Ensemble Learning) được xây dựng từ nhiều cây quyết định độc lập.

Ý tưởng chính của thuật toán là kết hợp kết quả dự đoán của nhiều cây nhằm giảm phương sai và nâng cao khả năng tổng quát hóa.

3.3.2. Lấy mẫu Bootstrap (Bootstrap Sampling)

Đối với mỗi cây quyết định, thuật toán tạo một tập dữ liệu huấn luyện mới bằng phương pháp lấy mẫu có hoàn lại (Sampling with Replacement).

Nhờ đó, mỗi cây được huấn luyện trên một tập dữ liệu khác nhau, tạo ra tính đa dạng trong toàn bộ mô hình.

3.3.3. Lựa chọn thuộc tính ngẫu nhiên (Feature Randomness)

Tại mỗi nút phân chia, thuật toán không sử dụng toàn bộ thuộc tính hiện có mà chỉ lựa chọn ngẫu nhiên một tập con thuộc tính để xem xét.

Cơ chế này giúp:

- Giảm sự tương quan giữa các cây.
- Tăng tính đa dạng của mô hình.
- Hạn chế hiện tượng quá khớp.

3.3.4. Bỏ phiếu đa số (Majority Voting)

Trong giai đoạn dự đoán, mỗi cây đưa ra một kết quả phân loại riêng. Kết quả cuối cùng của Random Forest được xác định thông qua cơ chế bỏ phiếu đa số (Majority Voting), trong đó lớp nhận được nhiều phiếu bầu nhất sẽ được lựa chọn.

3.3.5. Xử lý mất cân bằng dữ liệu

Mô hình sử dụng tham số: **class_weight = 'balanced'**

Trọng số của mỗi lớp được tính theo công thức:

$$Weight_c = N_samples / (N_classes \times N_samples_c)$$

Cơ chế này giúp mô hình chú ý nhiều hơn đến các lớp có số lượng mẫu ít hơn, từ đó cải thiện khả năng phân loại trong trường hợp dữ liệu mất cân bằng.

Training Random Forest (ensemble)...

RandomForestClassifier	
Parameters	
n_estimators	200
criterion	'gini'
max_depth	15
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	'sqrt'
max_leaf_nodes	None
min_impurity_decrease	0.0
bootstrap	True
oob_score	False
n_jobs	-1
random_state	42
verbose	0
warm_start	False
class_weight	'balanced'
ccp_alpha	0.0
max_samples	None
monotonic_cst	None

3.3.6. Độ phức tạp tính toán và khả năng song song hóa

Do được xây dựng từ nhiều cây quyết định độc lập, mô hình Rừng ngẫu nhiên có chi phí tính toán cao hơn so với một cây quyết định đơn lẻ. Thời gian huấn luyện tăng theo số lượng cây được sử dụng trong mô hình.

Tuy nhiên, quá trình xây dựng các cây diễn ra độc lập với nhau, cho phép thực hiện huấn luyện song song trên nhiều lõi xử lý. Trong đề tài này, mô hình được cấu hình với tham số: **n_jobs = -1**

Nhằm sử dụng toàn bộ tài nguyên xử lý khả dụng của hệ thống. Điều này giúp rút ngắn thời gian huấn luyện mà vẫn duy trì hiệu năng của mô hình.

4. Kết Quả

--- Decision Tree ---				--- Random Forest ---			
Accuracy : 0.5819				Accuracy : 0.6368			
Precision: 0.5838				Precision: 0.6342			
Recall : 0.5766				Recall : 0.6504			
F1-score : 0.5802				F1-score : 0.6422			
precision recall f1-score support				precision recall f1-score support			
No Detection	0.58	0.59	0.58	No Detection	0.64	0.62	0.63
1996				1996			
Detection	0.58	0.58	0.58	Detection	0.63	0.65	0.64
2005				2005			
accuracy		0.58	4001	accuracy		0.64	4001
macro avg	0.58	0.58	0.58	macro avg	0.64	0.64	0.64
4001				4001			
weighted avg	0.58	0.58	0.58	weighted avg	0.64	0.64	0.64
4001				4001			
Confusion Matrix:				Confusion Matrix:			
[[1172 824]				[[1244 752]			
[849 1156]]				[701 1304]]			

